

El Cambio de Paradigma en el Diseño Web: Auge, Crisis y el Resurgimiento de Vanilla CSS frente al Dominio de Tailwind CSS

En la trayectoria histórica del desarrollo web, pocas herramientas han generado un debate tan polarizado y, al mismo tiempo, una adopción tan masiva como Tailwind CSS. Este framework, que introdujo un cambio de mentalidad radical hacia el enfoque de "utilidades primero" (utility-first), ha pasado de ser un experimento personal a convertirse en una infraestructura fundamental para millones de proyectos.¹ Sin embargo, al alcanzar el ecuador de la década de 2020, el ecosistema se encuentra en un punto de inflexión crítico. El avance sin precedentes de las capacidades nativas de CSS, sumado al impacto disruptivo de la Inteligencia Artificial en los modelos de negocio de código abierto, ha dado lugar a un sector creciente de profesionales que abogan por el retorno al CSS puro o "Vanilla CSS".² Este reporte analiza la génesis de Tailwind, sus beneficios operativos, las raíces técnicas del movimiento de resistencia y la reconfiguración del mercado hacia alternativas emergentes y estándares nativos.

Génesis y Evolución de Tailwind CSS: Del Fracaso a la Infraestructura Global

El origen de Tailwind CSS es un relato de innovación accidental y resolución pragmática de problemas. Adam Wathan, el arquitecto principal detrás del framework, no buscaba inicialmente crear un estándar de la industria, sino optimizar su propio flujo de trabajo.¹ Wathan, quien había abandonado la universidad para dedicarse a la música y la producción de sonido antes de regresar al desarrollo de software, comenzó a experimentar con metodologías de CSS mientras trabajaba en KiteTail, una aplicación diseñada para compartir artículos en equipo.¹

Durante el desarrollo de KiteTail en 2017, Wathan experimentó una frustración creciente con los frameworks de la época, particularmente Bootstrap.⁴ La transición de Bootstrap 4 de Less a Sass fue un factor determinante; Wathan prefería Less por su sintaxis y se vio obligado a escribir su propio CSS para mantener su flujo de trabajo habitual.⁴ En este proceso, notó que cada vez que iniciaba un nuevo proyecto, lo único que persistía eran pequeñas clases de utilidad para márgenes, rellenos y flexbox.⁴ Esta observación sembró la semilla de lo que se convertiría en una de las herramientas más influyentes del desarrollo front-end moderno.

El Surgimiento del Enfoque Utility-First

La colaboración con Steve Schoger, su socio comercial y diseñador, fue fundamental para refinar la estética y la funcionalidad del framework.¹ En reuniones informales en cafeterías,

ambos discutían cómo abstraer las decisiones de diseño en clases atómicas que permitieran construir interfaces personalizadas sin escribir CSS manual.⁴ Tailwind CSS se lanzó oficialmente en la noche de Halloween de 2017 como un proyecto de código abierto bajo la marca KiteTail, evolucionando rápidamente gracias a la transparencia de Wathan, quien compartía su proceso de desarrollo a través de transmisiones en vivo.¹

A diferencia de los frameworks de componentes tradicionales como Bootstrap o Foundation, que proporcionan botones y tarjetas pre-diseñadas, Tailwind ofreció clases de bajo nivel que actúan como "bloques de construcción" o piezas de LEGO.⁶ Este cambio filosófico permitió a los desarrolladores crear diseños únicos sin el estigma de que todos los sitios web parecieran iguales, un problema común en la era de Bootstrap.⁹

Hitos en la Historia de Tailwind CSS	Año	Impacto Tecnológico
Desarrollo Inicial de KiteTail	2017	Origen de las utilidades atómicas en Less. ⁴
Lanzamiento Público v0.1.0	2017	Introducción del concepto utility-first a la comunidad. ¹
Fundación de Tailwind Labs	2019	Profesionalización del mantenimiento y ecosistema. ⁵
Introducción del Motor JIT	2021	Generación de CSS en tiempo real, eliminando el purgado lento. ⁶
Lanzamiento de Tailwind v4.0	2025	Motor Oxide ultra rápido y variables CSS nativas. ¹²
Crisis Financiera por IA	2026	Despido del 75% del equipo de ingeniería por caída de ingresos. ¹⁵

Mecanismos de Rendimiento y el Motor JIT

Uno de los mayores avances técnicos en la historia del framework fue la introducción del compilador Just-In-Time (JIT).⁶ En sus versiones iniciales, Tailwind generaba un archivo CSS masivo que contenía todas las clases posibles, las cuales debían ser eliminadas mediante herramientas como PurgeCSS en el proceso de producción para evitar el bloat.⁶ El motor JIT,

que se convirtió en el estándar con la versión 3, revolucionó esto al escanear los archivos de plantilla (HTML, JSX, Vue) y generar instantáneamente solo las clases que el desarrollador está utilizando.⁶

Este enfoque no solo redujo drásticamente el tamaño de los paquetes de CSS en producción —llevándolos frecuentemente a menos de 10kb—, sino que también habilitó el uso de valores arbitrarios y variantes complejas, como estados de hover dinámicos y breakpoints responsivos integrados directamente en el nombre de la clase.¹⁷ En 2025, el lanzamiento de la versión 4.0 con el motor Oxide llevó el rendimiento a nuevos límites, logrando builds completas hasta 5 veces más rápidas y builds incrementales más de 100 veces más veloces.¹³

Beneficios Estratégicos: Por qué el Mercado Adoptó Tailwind

La adopción de Tailwind CSS no se debió únicamente a su novedad técnica, sino a la resolución de problemas estructurales en la gestión de estilos a gran escala. La industria del software valora la predictibilidad y la velocidad, dos áreas donde el CSS tradicional a menudo presenta desafíos.¹¹

Aceleración del Flujo de Trabajo y Reducción del Context-Switching

El beneficio más citado por los profesionales es la eliminación del cambio de contexto.¹⁷ En un flujo de trabajo tradicional, un desarrollador debe alternar constantemente entre su archivo de marcado y su hoja de estilos, inventando nombres de clases semánticos que a menudo se vuelven irrelevantes o confusos con el tiempo.¹⁸ Tailwind permite que el estilo viva junto al elemento, lo que facilita una iteración visual inmediata.²

Esta co-ubicación de preocupaciones ha demostrado ser especialmente efectiva en entornos de desarrollo basados en componentes como React, Vue y Svelte.⁸ Al encapsular los estilos dentro del componente, se eliminan los riesgos de "fugas de estilo" (style leaks), donde una regla definida para una sección del sitio afecta involuntariamente a otra debido a la cascada global de CSS.¹⁸

Consistencia a Través de Tokens de Diseño

Tailwind actúa como un sistema de diseño empaquetado. A través de su archivo de configuración, los equipos pueden definir una paleta de colores, escalas de espaciado y tipografías estrictas.¹⁰ Esto evita la proliferación de valores arbitrarios como margin-top: 17px en un lugar y 1rem en otro; en su lugar, los desarrolladores se ven obligados a usar tokens como mt-4, lo que garantiza una armonía visual automática en toda la aplicación.¹²

Optimización Automática y Rendimiento en el Cliente

Desde una perspectiva de rendimiento, Tailwind rompe la tendencia de crecimiento lineal de las hojas de estilo.⁷ En proyectos grandes de CSS tradicional, los archivos tienden a crecer con

cada nueva característica, ya que los desarrolladores temen eliminar clases antiguas por miedo a romper partes desconocidas del sitio.⁷ Con Tailwind, el tamaño del CSS se estabiliza rápidamente porque las clases de utilidad se reutilizan en lugar de crear nuevas reglas, lo que resulta en tiempos de carga más rápidos y mejores puntuaciones en Core Web Vitals.⁷

Métrica de Rendimiento	CSS Tradicional / Bootstrap	Tailwind CSS (JIT/Oxide)
Tamaño de CSS en Producción	100kb - 500kb+	5kb - 20kb. ⁷
Crecimiento del Código	Lineal con la funcionalidad	Logarítmico/Estable. ⁷
Tiempo de Análisis del Navegador	Mayor debido a selectores complejos	Mínimo debido a clases atómicas. ¹⁷
Velocidad de Compilación (Incremental)	Variable	<10ms en v4.0. ¹³

El Surgimiento de la Resistencia: Analizando el Sector que Evita Tailwind

A pesar de su dominio —alcanzando un 51% de adopción en 2025 y más de 75 millones de descargas mensuales—, existe un movimiento creciente de desarrolladores que abogan por alejarse del framework.² Esta resistencia no es monolítica, sino que se compone de diversos argumentos técnicos y filosóficos que han ganado tracción a medida que el lenguaje CSS nativo ha madurado.¹³

La Crítica a la "Sopa de Clases" y la Legibilidad del HTML

El argumento más común contra Tailwind es la degradación estética y funcional del marcado HTML.¹⁰ Una simple tarjeta de usuario puede terminar con una cadena de 20 o más clases de utilidad, lo que dificulta la lectura del código y el escaneo de la estructura del documento.¹⁹ Para muchos veteranos de la industria, esto representa una violación del principio de separación de conceptos, devolviendo el desarrollo web a la era de los estilos en línea que se consideraban una mala práctica hace dos décadas.¹⁹

Este fenómeno, a menudo llamado "class soup", genera una fricción particular en proyectos donde la limpieza del código y la accesibilidad son prioridades máximas.¹⁰ Desarrolladores en foros como Reddit y comunidades como la de Midudev han señalado que, si bien Tailwind es rápido para el prototipado inicial, el mantenimiento a largo plazo de estas cadenas de clases

puede volverse una pesadilla técnica si no se gestionan adecuadamente mediante abstracciones de componentes.¹⁹

La Dependencia de Herramientas y el Bloqueo de Proveedor

Un sector de la comunidad se muestra reacio a depender de un paso de compilación pesado y de una herramienta de terceros para una tarea tan fundamental como el estilo.¹⁹ El uso de Tailwind crea un fuerte "vendor lock-in"; una vez que miles de archivos están inundados con sintaxis específica del framework, la migración a cualquier otra solución se vuelve económicamente inviable.⁸

Además, la necesidad de configurar PostCSS, compiladores y plugins añade complejidad innecesaria a proyectos pequeños o estáticos que podrían resolverse simplemente con un archivo .css nativo.² Esta "fatiga de herramientas" ha llevado a muchos a redescubrir la elegancia de no tener dependencias externas.²

La Evolución de las Capacidades Nativas de CSS

Quizás el factor más poderoso que impulsa la vuelta a Vanilla CSS es que el propio CSS se ha vuelto "bueno, realmente bueno".² Muchas de las ventajas competitivas que Tailwind ofrecía en 2017 han sido absorbidas por el estándar de la plataforma.² La introducción de características modernas ha reducido drásticamente la cantidad de código necesario para lograr diseños complejos.

Característica de Tailwind	Equivalente en Vanilla CSS Moderno (2024-2026)
Breakpoints responsivos	Container Queries (Consultas de Contenedor). ²
Evitar colisiones de nombres	CSS Nesting (Anidamiento) nativo y @scope. ²
Paletas dinámicas	Relative Color Syntax y color-mix(). ²
Estilos basados en contexto	Selector :has() (el "selector de padres"). ²
Gestión de especificidad	Cascade Layers (@layer). ¹²
Lógica condicional	Función if() nativa (incorporación 2026). ³²

La Revolución de Vanilla CSS: Por qué el Estándar

está Ganando Terreno

El resurgimiento de Vanilla CSS no es un retroceso, sino una evolución hacia un flujo de trabajo más ligero y potente que aprovecha la "superficie de la plataforma".² A partir de 2025, el soporte de los navegadores para características avanzadas superó el 90% del tráfico web en mercados clave, permitiendo que los desarrolladores abandonen los preprocesadores y frameworks pesados.²

El Poder de las Container Queries

Históricamente, la web responsiva se ha basado en el ancho de la ventana gráfica (viewport) a través de media queries.² Tailwind simplificó esto con prefijos como md: o lg:.⁶ Sin embargo, el diseño moderno basado en componentes exige que un elemento sepa cuánto espacio tiene dentro de su contenedor, no en la pantalla completa.²

Las Container Queries permiten que un componente de "tarjeta", por ejemplo, cambie de un diseño vertical a uno horizontal basándose en si está en una barra lateral estrecha o en el área de contenido principal.² Lograr esto con Tailwind requiere lógica de JavaScript compleja o clases condicionales manuales, mientras que en Vanilla CSS es una declaración nativa de pocas líneas.²

El Selector :has() y la Inteligencia del Layout

El selector :has() ha sido calificado como el "Santo Grial" del CSS.² Permite aplicar estilos a un elemento padre en función de si contiene ciertos hijos o estados.²⁹ Por ejemplo, se puede añadir un borde especial a una tarjeta solo si contiene una imagen, o resaltar un formulario completo si tiene campos inválidos, todo sin una sola línea de JavaScript.²⁹ Esta capacidad de introspección nativa hace que muchas de las utilidades condicionales de Tailwind parezcan verbosas y redundantes.²

Tipografía y Color Dinámico

Vanilla CSS ha introducido herramientas de diseño que superan las capacidades de configuración estática de Tailwind.² La tipografía fluida mediante la función clamp() permite que los textos escalen suavemente entre un mínimo y un máximo sin necesidad de múltiples media queries.³¹ Asimismo, el uso de colores relativos permite derivar variaciones de un color base (como sombras o luces) dinámicamente en el navegador, en lugar de generar cientos de clases de colores en el archivo de configuración.²

El Impacto de la Inteligencia Artificial: La Crisis de Tailwind Labs en 2026

Un factor inesperado que ha alterado profundamente el panorama de Tailwind es la Inteligencia Artificial generativa. En 2025 y 2026, la IA se convirtió tanto en el mayor aliado de la adopción

de Tailwind como en el verdugo de su sostenibilidad financiera.³

La Paradoja de la Adopción masiva y el Colapso de Ingresos

Tailwind es el lenguaje perfecto para los modelos de lenguaje (LLM) como Claude, ChatGPT y Gemini.² Debido a que los estilos están en el mismo archivo que el marcado, la IA tiene todo el contexto necesario para generar componentes completos y funcionales con una precisión asombrosa.² Esto ha llevado a que Tailwind sea considerado la "infraestructura" por defecto para herramientas de codificación asistida como Cursor y v0.dev.²

Sin embargo, esta facilidad de uso rompió el modelo de negocio de Tailwind Labs. La empresa dependía de un embudo de ventas donde los desarrolladores visitaban la documentación oficial, aprendían el framework y eventualmente compraban productos premium como Tailwind UI (plantillas de pago).³ Con la llegada de la IA, los desarrolladores dejaron de visitar el sitio web; las dudas se resuelven en el chat del IDE y los componentes se generan por prompt.¹⁵

Métrica de Crisis en Tailwind Labs (Enero 2026)	Cambio Porcentual	Causa Identificada
Tráfico a la Documentación Oficial	-40%	Desintermediación por IA. ³
Ingresos por Venta de Productos Premium	-80%	Los usuarios ya no ven los anuncios en la web. ¹⁵
Personal de Ingeniería (Layoffs)	-75%	Inestabilidad financiera severa. ³
Descargas de npm	+30% (Récord)	Uso masivo como infraestructura impulsada por IA. ¹⁵

El Incidente del /lms.txt y la Reacción de la Comunidad

El punto crítico de esta crisis ocurrió en enero de 2026, cuando Adam Wathan declinó públicamente una solicitud de pull request para añadir un archivo /lms.txt a la documentación.¹⁵ Wathan explicó que facilitar que los modelos de IA consumieran su contenido solo aceleraría la destrucción de su negocio, revelando que el día anterior había tenido que despedir a 3 de sus 4 ingenieros.¹⁵ Este incidente generó un debate global sobre la ética del scraping de datos por parte de las empresas de IA y la viabilidad de los modelos de negocio basados en documentación abierta.³⁶ La supervivencia de Tailwind como proyecto mantenido

profesionalmente se vio amenazada hasta que empresas como Vercel y Google AI Studio intervinieron con patrocinios de emergencia.²

Alternativas Modernas: Más allá de Tailwind y Vanilla CSS

Para aquellos que buscan los beneficios de un framework pero desean evitar las desventajas de Tailwind o la verbosidad de Vanilla CSS, han surgido alternativas que redefinen la relación entre el estilo y el código.¹⁴

Panda CSS: Seguridad de Tipos y Rendimiento Nativo

Panda CSS se ha posicionado como el competidor más fuerte en el sector empresarial.¹⁴ Desarrollado por el equipo de Chakra UI, ofrece una experiencia de "CSS-in-JS" que se compila a CSS estático en tiempo de construcción.¹⁴ Su mayor ventaja es la seguridad de tipos total (Type-safety), permitiendo que los desarrolladores reciban errores en su editor si intentan usar un valor de diseño no permitido, algo que Tailwind solo logra mediante plugins externos.¹⁴

StyleX: La Solución de Meta para la Escalabilidad

Originalmente creado por Meta para reconstruir Facebook, StyleX es un compilador de estilos que genera CSS atómico altamente optimizado.⁴¹ A diferencia de Tailwind, StyleX no usa clases de utilidad en el marcado, sino objetos de JavaScript, lo que facilita la mantenibilidad en bases de código gigantescas donde las colisiones de estilos son un riesgo constante.⁴¹ En 2026, StyleX es la opción predilecta para equipos que priorizan la arquitectura robusta sobre la velocidad de prototipado inicial.⁴¹

UnoCSS: El Motor Atómico de Próxima Generación

UnoCSS no se considera un framework, sino un "motor" que puede emular cualquier sistema de utilidades.⁴³ Es significativamente más rápido que Tailwind y permite crear reglas personalizadas con una facilidad asombrosa.⁴³ Al ser totalmente bajo demanda, no genera ni un solo byte de CSS que no esté estrictamente en uso, lo que lo hace ideal para micro-frontends y aplicaciones de rendimiento crítico.⁴³

El Modelo "shadcn/ui": La Abstracción Superior

Una tendencia masiva ha sido el movimiento hacia bibliotecas de componentes como **shadcn/ui**.¹⁴ Estas no son frameworks CSS per se, sino colecciones de componentes que utilizan Tailwind por debajo, pero que el desarrollador simplemente copia y pega en su repositorio.³⁸ Esto resuelve la crítica de la "sopa de clases" al ocultar las utilidades dentro de componentes con nombres semánticos como `<Button />` o `<Dialog />`, combinando lo mejor de ambos mundos.¹⁴

Categoría	Ejemplo Principal	Fortaleza	Debilidad
Framework de Utilidades	Tailwind CSS	Ecosistema masivo y velocidad. ¹⁰	Legibilidad del HTML. ¹⁹
CSS-in-JS Estático	Panda CSS	Seguridad de tipos y DX. ¹⁴	Curva de aprendizaje avanzada. ¹⁴
Motor Atómico	UnoCSS	Rendimiento y flexibilidad. ⁴³	Menor ecosistema de plugins. ⁴⁰
Compilador Empresarial	StyleX	Mantenibilidad a escala extrema. ⁴¹	Verboso y orientado a React. ⁴¹
Componentes de Copia	shadcn/ui	Rapidez de UI y accesibilidad. ¹⁴	Requiere gestionar el código copiado. ³⁸

Análisis Regional: España y Latinoamérica (2025-2026)

En el mercado hispanohablante, el debate entre Tailwind y Vanilla CSS ha sido especialmente vibrante. Comunidades en España y Latinoamérica muestran una división generacional y de escala de proyectos.⁸

El Sector del Prototipado y las Startups

En los ecosistemas emprendedores de Madrid, Ciudad de México y Buenos Aires, Tailwind CSS sigue siendo la herramienta indiscutible.⁸ La presión por lanzar productos mínimos viables (MVPs) rápidamente hace que la velocidad de desarrollo de Tailwind sea insustituible.⁸ Consultoras tecnológicas en estas regiones informan que el uso de Tailwind reduce el tiempo de desarrollo de interfaces en hasta un 300% en comparación con el CSS manual.¹⁰

La Resistencia en Agencias de Diseño y Freelancers

Por otro lado, un sector de agencias boutique y desarrolladores freelance que se especializan en sitios web "artesanales" o de alta fidelidad visual están liderando el retorno a Vanilla CSS.⁸ Estos profesionales argumentan que Tailwind "homogeniza" la web, haciendo que todos los diseños se sientan similares debido a las restricciones de su sistema de tokens.⁸ Para proyectos donde el diseño es el principal diferenciador, el control absoluto de Vanilla CSS es visto como una ventaja competitiva.²²

Formación y Empleabilidad

Curiosamente, las encuestas en comunidades locales indican que, aunque Tailwind es la habilidad más demandada en las ofertas de trabajo actuales, existe una preocupación creciente de que los nuevos desarrolladores (Juniors) estén "olvidando" cómo funciona el CSS real.²² Esto ha llevado a un resurgimiento de cursos de "CSS Moderno" que enseñan Grid, Flexbox y las nuevas APIs de 2026 como base fundamental antes de introducir cualquier framework.¹²

Conclusiones Estratégicas y Recomendaciones

El análisis de la trayectoria de Tailwind CSS y la tendencia hacia Vanilla CSS permite extraer conclusiones matizadas sobre el futuro del diseño web. No estamos ante la muerte de Tailwind, sino ante su transformación en una capa de infraestructura que a menudo será gestionada por máquinas (IA) o escondida tras abstracciones de componentes.²

Síntesis del Estado Actual

1. **Tailwind es Infraestructura, no solo un Framework:** Su popularidad ha alcanzado un nivel donde es la lengua franca de la web moderna, especialmente para la IA.² Sin embargo, su sostenibilidad económica depende de una reconfiguración total de su modelo de negocio, alejándose del tráfico de documentación y acercándose a las licencias empresariales o patrocinios.¹⁶
2. **Vanilla CSS es la Nueva Vanguardia:** Lejos de ser "antiguo", el CSS nativo de 2026 es más potente que nunca.² Características como las Container Queries y el selector :has() eliminan la necesidad de muchas de las abstracciones que antes justificaban el uso de frameworks.²
3. **La IA ha cambiado el "Costo" del Estilo:** Si la IA puede escribir CSS nativo con la misma facilidad que Tailwind, el beneficio de la velocidad de escritura manual de Tailwind disminuye para los desarrolladores solitarios, pero se mantiene como una ventaja para la consistencia en equipos grandes.²

Recomendaciones para la Toma de Decisiones

- **Elegir Tailwind CSS cuando:** Se trabaje en equipos medianos o grandes que necesitan una convención compartida; se busque una integración perfecta con herramientas de IA para prototipado rápido; o se necesite un sistema de diseño consistente y probado.¹¹
- **Elegir Vanilla CSS cuando:** El rendimiento crítico y la reducción de dependencias sean la prioridad absoluta; el proyecto requiera un control artístico total que desafíe los sistemas de tokens; o se desee una base de código con la máxima longevidad posible que no dependa de compiladores externos.²
- **Considerar Alternativas (Panda/StyleX) cuando:** Se esté construyendo una aplicación empresarial de gran escala donde la seguridad de tipos o la prevención de colisiones de estilos sea un requisito de ingeniería innegociable.¹⁴

El futuro del diseño web no será una victoria total de uno sobre otro, sino una convivencia pragmática. La tendencia real es el uso de un "enfoque híbrido": utilizar Tailwind para el 90% de la interfaz "aburrida" y consistente, y aplicar Vanilla CSS moderno para el 10% de interacciones raras, animaciones complejas o layouts inteligentes basados en contenedores.¹⁹ En esta nueva era, el conocimiento profundo de los estándares de la plataforma web sigue siendo el activo más valioso para cualquier desarrollador front-end.

Works cited

1. Founder Story: Adam Wathan of Tailwind CSS | Frederick AI, accessed May 5, 2026, <https://www.frederick.ai/blog/adam-wathan-tailwind-css>
2. I Stopped Using Tailwind. The Reason Has Nothing to Do With Tailwind. | by Mutasim Billah Toha | JavaScript in Plain English, accessed May 5, 2026, <https://javascript.plainenglish.io/i-stopped-using-tailwind-the-reason-has-nothing-to-do-with-tailwind-f67867227b53>
3. Tailwind CSS AI Impact: Docs Traffic Collapse Triggers 75% Engineering Layoffs - remio AI, accessed May 5, 2026, <https://www.remio.ai/post/tailwind-css-ai-impact-docs-traffic-collapse-triggers-75-engineering-layoffs>
4. The Story of Tailwind CSS feat. Adam Wathan - OfferZen, accessed May 5, 2026, <https://www.offerzen.com/blog/adam-wathan-story-tailwind-css>
5. S3 E9: Adam Wathan, Tailwind CSS - Apple Podcasts, accessed May 5, 2026, <https://podcasts.apple.com/tr/podcast/s3-e9-adam-wathan-tailwind-css/id1466861744?i=1000491305315>
6. Tailwind CSS - Wikipedia, accessed May 5, 2026, https://en.wikipedia.org/wiki/Tailwind_CSS
7. Why Use Tailwind CSS for Your Next Project, accessed May 5, 2026, <https://jifelix.info/blog/why-use-tailwind-css-for-your-next-project>
8. La Eterna Batalla: ¿Tailwind CSS o Vanilla CSS? Tu Guía para Elegir al Ganador - Medium, accessed May 5, 2026, <https://medium.com/@osvaocampo/la-eterna-batalla-tailwind-css-o-vanilla-css-tu-gu%C3%ADa-para-elegir-al-ganador-630b4eda7a5d>
9. The End of Traditional CSS: Why Tailwind Usage Surpassed Bootstrap in 2024 - Medium, accessed May 5, 2026, https://medium.com/@sohail_saifii/the-end-of-traditional-css-why-tailwind-usage-surpassed-bootstrap-in-2024-fb075a854401
10. Tailwind CSS: qué es y cómo usarlo en tus proyectos (2026) - ThePower Education, accessed May 5, 2026, <https://thepower.education/blog/tech/tailwind-css-simplifica-tu-diseno-web>
11. Tailwind CSS Breakdown: How It Works, Key Advantages, and Top Alternatives, accessed May 5, 2026, <https://shivlab.com/blog/tailwind-css-everything-you-need-to-know-about/>
12. It's almost 2026: Why Are We Still Arguing About CSS vs Tailwind - DEV Community, accessed May 5, 2026,

<https://dev.to/toboreeeee/its-almost-2026-why-are-we-still-arguing-about-css-vs-tailwind-291f>

13. Top CSS Frameworks in 2025: What Devs Actually Use - Tailkits, accessed May 5, 2026, <https://tailkits.com/blog/popular-css-frameworks/>
14. The Best CSS Framework for Modern Web Development (2026 Guide), accessed May 5, 2026, <https://www.dataenriche.com/best-css-framework-modern-web-development/>
15. Tailwind's AI Crisis: 75% Layoffs, 80% Revenue Drop - Leanware, accessed May 5, 2026, <https://www.leanware.co/insights/tailwind-ai-crisis>
16. Tailwind CSS Announces 75% Layoffs as LLMs Reshape OSS Business Models, accessed May 5, 2026, <https://socket.dev/blog/tailwind-css-announces-layoffs>
17. 6 reasons to use Tailwind over traditional CSS | npm run dev - WordPress.com, accessed May 5, 2026, <https://npmrundev.wordpress.com/2021/08/04/6-reasons-to-use-tailwind-over-traditional-css/>
18. 6 reasons to use Tailwind over traditional CSS - DEV Community, accessed May 5, 2026, <https://dev.to/charliejoel/6-reasons-to-use-tailwind-over-traditional-css-1nc3>
19. Why do some people prefer Tailwind CSS over CSS? : r/webdev - Reddit, accessed May 5, 2026, https://www.reddit.com/r/webdev/comments/1qtpqkg/why_do_some_people_prefer_tailwind_css_over_css/
20. Best CSS Frameworks 2026: Top Styling Tools Every Frontend Developer Should Know, accessed May 5, 2026, <https://quartzdevs.com/resources/best-css-frameworks-2026-top-styling-tools-every-frontend-developer-should-know>
21. 6 Best CSS Frameworks for Developers in 2026, accessed May 5, 2026, <https://www.valoremreply.com/resources/insights/blog/2025/april/6-best-css-frameworks-for-developers-in-2026/>
22. Tailwind CSS vs. Vanilla CSS: Which One Should You Choose? - DEV Community, accessed May 5, 2026, https://dev.to/kawthar_abolade/tailwind-css-vs-vanilla-css-which-one-should-you-choose-cno
23. ¿Por qué algunas personas prefieren Tailwind CSS sobre CSS? : r/webdev - Reddit, accessed May 5, 2026, https://www.reddit.com/r/webdev/comments/1qtpqkg/why_do_some_people_prefer_tailwind_css_over_css/?tl=es-419
24. Introduction to Tailwind CSS: A Comprehensive Overview | by Cchinanuekpere | Medium, accessed May 5, 2026, <https://medium.com/@cchinanuekpere0505/introduction-to-tailwind-css-a-comprehensive-overview-c89385d67300>
25. Why Tailwind CSS is a Game-Changer for Developers - Devōt, accessed May 5, 2026, <https://devot.team/blog/tailwind-css-tutorial>
26. Tailwind CSS crisis 2026: what developers need to know - Workspace, accessed May 5, 2026,

- <https://workspace.hr/blog/tailwind-css-crisis-2026-what-developers-need-to-know>
27. Are there developers who still don't prefer Tailwind CSS as their first ..., accessed May 5, 2026,
https://www.reddit.com/r/Frontend/comments/1qwclql/are_there_developers_who_still_dont_prefer/
 28. TailwindCSS despide a 3/4 de sus ingenieros luego de que la IA arruinara su modelo de negocios. : r/devsarg - Reddit, accessed May 5, 2026,
https://www.reddit.com/r/devsarg/comments/1q7cfcb/tailwindcss_despide_a_34_d_e_sus_ingenieros_luego/
 29. Latest CSS Features in 2024-2025: What's New and Exciting? - DEV Community, accessed May 5, 2026,
<https://dev.to/prathamisonline/latest-css-features-in-2024-2025-whats-new-and-exciting-3oji>
 30. Using container size and style queries - CSS - MDN Web Docs, accessed May 5, 2026,
https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_size_and_style_queries
 31. Essential Modern CSS Features for 2026 - DEV Community, accessed May 5, 2026, <https://dev.to/digitalunicon/essential-modern-css-features-for-2026-2j1k>
 32. 2026 CSS Features You Must Know (Shipped Late 2025–Now ..., accessed May 5, 2026, <https://blog.riadkilani.com/2026-css-features-you-must-know/>
 33. CSS Usage - State of CSS 2025, accessed May 5, 2026,
<https://2025.stateofcss.com/en-US/usage/>
 34. Is anyone actually using @container queries in production? : r/css - Reddit, accessed May 5, 2026,
https://www.reddit.com/r/css/comments/1sdpeaj/is_anyone_actually_using_container_queries_in/
 35. Tailwind future - RapidWeaver Elements Forum, accessed May 5, 2026,
<https://forums.realmacsoftware.com/t/tailwind-future/53585>
 36. TailwindCSS laid off 75% of their developers? | by Alexis Brochu - Medium, accessed May 5, 2026,
<https://medium.com/design-bootcamp/tailwindcss-laid-off-75-of-their-developers-f98f04b2f16d>
 37. El equipo de Tailwind css se va a la mierda. ¿Qué pasaría cuando eso ocurra? - Reddit, accessed May 5, 2026,
https://www.reddit.com/r/learnprogramming/comments/1qdfahm/tailwind_css_team_is_going_under_what_would/?tl=es-es
 38. Compare Tailwind CSS Alternatives (2026) - Product Hunt, accessed May 5, 2026, <https://www.producthunt.com/products/tailwind-css/alternatives>
 39. 10+ Best Tailwind CSS Alternatives In 2026 - FlyonUI, accessed May 5, 2026,
<https://flyonui.com/blog/tailwind-css-alternatives/>
 40. Tailwind CSS vs Panda CSS: Complete Comparison Guide 2026 | FastBuilder.AI Blog, accessed May 5, 2026, <https://fastbuilder.ai/blog/tailwindcss-vs-panda-css>
 41. StyleX vs. Tailwind: Meta's Take on CSS-in-JS Maintainability - The New Stack, accessed May 5, 2026,

- <https://thenewstack.io/stylex-vs-tailwind-metas-take-on-css-in-js-maintainability/>
42. tailwind v4 vs stylex : r/nextjs - Reddit, accessed May 5, 2026,
https://www.reddit.com/r/nextjs/comments/1r0znqx/tailwind_v4_vs_stylex/
 43. Top 13 Tailwind CSS Alternatives For Developers, accessed May 5, 2026,
<https://cssauthor.com/tailwind-css-alternatives-for-developers/>
 44. tinyhref/tailwind-alternatives: A curated collection of the best alternatives to Tailwind CSS - GitHub, accessed May 5, 2026,
<https://github.com/tinyhref/tailwind-alternatives>
 45. 10 Best Tailwind CSS Alternatives: Every Developer Should Know (2024),
accessed May 5, 2026, <https://snappify.com/blog/tailwind-css-alternatives>